

UNIVERSIDAD DISTRITAL FRANCISCO JOSÉ DE CALDAS

FACULTAD DE INGENIERÍA

PROYECTO CURRICULAR DE INGENIERÍA DE SISTEMAS

Market Prediction System (MPS)

Especificación de Requerimientos y Diseño Arquitectural de Software

Estándar ISO/IEC/IEEE 29148 / 830

Integrantes:

Sebastián Avendaño Rodríguez — Cód. 20232020101
Juan Sebastián Rodríguez Serrano — Cód. 20232020038
Manuel Arturo Rueda Guiza — Cód. 20232020052

Asignatura: Fundamentos de Ingeniería de Software

Docente: Jose Martin Delgado

Bogotá D.C., Colombia

2026

Índice

1. Introducción y Justificación del Problema	2
1.1. Planteamiento del Problema	2
1.2. Justificación de la Solución: Market Prediction System (MPS)	2
2. Marco Metodológico de Desarrollo	2
3. Arquitectura de Software: Arquitectura Hexagonal	3
4. Prototipado y Diseño de Interfaz (Wireframes)	3
5. Especificación Detallada de Casos de Uso (UML 2.5)	6
5.1. UC-01 — Ingresar Intención de Inversión	6
5.2. UC-02 — Ver Recomendación de Inversión	7
5.3. UC-03 — Consultar Explicación (“¿Por qué?”)	7
5.4. UC-04 — Visualizar Indicador de Confianza	8
5.5. UC-05 — Visualizar Contexto Macroeconómico	8
5.6. UC-06 — Ajustar Distribución de Capital	8
5.7. UC-07 — Visualizar Estrategias de Salida (Stop-loss / Take-profit)	9
5.8. UC-08 — Consultar Hub Educativo	9
5.9. UC-09a — Registrarse en la Plataforma	10
5.10. UC-09b — Configurar Alertas de Ticker	10
5.11. UC-10 — Consultar Historial de Alertas	10
6. Catálogo de Requerimientos de Software (SRS)	11
6.1. Requerimientos Funcionales (RF)	11
6.2. Requerimientos No Funcionales (RNF)	14
6.3. Matriz de Trazabilidad Requerimientos vs. Historias de Usuario	17
7. Modelado Estructural y del Comportamiento (Diagramas UML / DER)	17
7.1. Diagrama de Casos de Uso	17
7.2. Diagrama de Contexto del Sistema	18
7.3. Diagramas de Secuencia	19
7.4. Diagramas de Flujo de Proceso	21
7.5. Modelo Entidad-Relación (MER)	21
7.6. Diagrama de Clases (UML)	22

1 Introducción y Justificación del Problema

1.1 Planteamiento del Problema

La toma de decisiones de inversión en los mercados de capitales modernos ha quedado reservada casi exclusivamente a firmas de comercio cuantitativo, fondos de cobertura y analistas técnicos avanzados. Estos actores disponen de acceso a terminales de datos financieros de baja latencia y algoritmos predictivos complejos fundamentados en aprendizaje profundo y series temporales.

Por el contrario, el inversor minorista común o la persona sin formación financiera se enfrenta a tres barreras fundamentales:

1. **Asimetría de información y sobrecarga cognitiva:** La avalancha de información desestructurada, cotizaciones crudas y jerga técnica impide discernir señales estadísticas sólidas de simple ruido de mercado.
2. **Caja negra algorítmica y desconfianza:** La mayoría de los sistemas que prometen predicciones no proporcionan explicabilidad sobre sus recomendaciones, impidiendo entender los fundamentos matemáticos de las señales emitidas.
3. **Riesgo ético y desinformación:** Múltiples aplicaciones promueven falsas promesas de rendimiento mediante aseveraciones deterministas categóricas (p. ej., “la acción subirá”), ignorando la naturaleza estocástica del mercado bursátil.

1.2 Justificación de la Solución: Market Prediction System (MPS)

El proyecto **Market Prediction System (MPS)** nace ante la necesidad de democratizar el análisis cuantitativo y el acceso a recomendaciones informadas para usuarios noveles o no profesionales de las finanzas. MPS cierra esta brecha mediante la integración de:

- **Interacción conversacional guiada:** Abstracción del ingreso tradicional mediante una barra de búsqueda en lenguaje natural, traduciendo intenciones patrimoniales en consultas computables.
- **Ocultamiento progresivo de complejidad (*Complexity Hiding*):** La interfaz presenta sugerencias directas y comprensibles, dejando la matemática estocástica pesada (matrices de covarianza, p -values, tensores neuronales) confinada en paneles de segundo nivel y en un Hub Educativo transversal.
- **Garantía de significancia estadística:** Cada señal de mercado generada por modelos neuronales profundos (MASTER y Galformer) es depurada mediante un test de hipótesis estadística (Z -test con $\alpha = 0,05$), garantizando que únicamente recomendaciones con sustento matemático sean emitidas.
- **Transparencia ética y regulatoria:** El sistema prohíbe el uso de lenguaje categórico futuro, adoptando marcos probabilísticos condicionales y especificando formalmente que el indicador de confianza no refleja una promesa de retorno financiero.

2 Marco Metodológico de Desarrollo

Para la ejecución del proyecto se seleccionó la metodología ágil **Kanban**. Dicha elección se fundamenta en los siguientes criterios operativos del equipo:

- **Gestión continua de flujo basada en Backlog:** Permite desacoplar la carga de trabajo en ítems atómicos derivados de las Historias de Usuario (HU), alimentando el flujo conforme se liberan capacidades técnicas.
- **Flexibilidad y ritmo individualizado:** Dado que el sistema combina componentes heterogéneos (modelos de Deep Learning en Python, microservicios asíncronos en Java y renderizado reactivo en el cliente web), Kanban facilita que cada desarrollador progrese a su propio ritmo sin el bloqueo que imponen las iteraciones de tiempo fijo (sprints rígidos).
- **Límites de Trabajo en Progreso (WIP Limits):** Restringe la cantidad de requerimientos funcionales en implementación simultánea, asegurando que componentes críticos como el pipeline de ingesta o el test estadístico alcancen una validación exhaustiva antes de pasar al despliegue.

3 Arquitectura de Software: Arquitectura Hexagonal

El sistema implementa el patrón de **Arquitectura Hexagonal** (Puertos y Adaptadores), motivado por la alta volatilidad tecnológica de los mercados y la naturaleza analítica y predictiva de la solución:

- **Aislamiento del Dominio Cuantitativo:** El núcleo del negocio (modelos econométricos, matrices de asignación de capital, cálculo de stop-loss/take-profit y filtros de hipótesis Z-score) reside en el centro de la arquitectura, aislado por completo de frameworks externos, bases de datos o motores de mensajería.
- **Intercambiabilidad de Infraestructura (Adaptadores):** Los puertos permiten desacoplar el consumo de datos de mercado (WebSockets externos), los brokers de señalización (Redis Pub/Sub) y los motores de persistencia (PostgreSQL para auditoría relacional y MongoDB para streaming de tensores). Esto asegura que cambios en el proveedor de streaming o en los motores de base de datos no alteren las reglas de negocio.
- **Testabilidad sin Infraestructura:** Permite verificar la lógica pura de negocio mediante dobles de prueba (*fakes* en memoria) sin requerir el despliegue de contenedores de persistencia en tiempo de compilación.

4 Prototipado y Diseño de Interfaz (Wireframes)

En esta sección se presentan los esquemas de prototipado de la interfaz de usuario, elaborados en herramientas de diseño de interacción (Figma/Balsamiq).



Figura 1: Prototipo 1: Vista Principal y Barra Conversacional de Intenciones

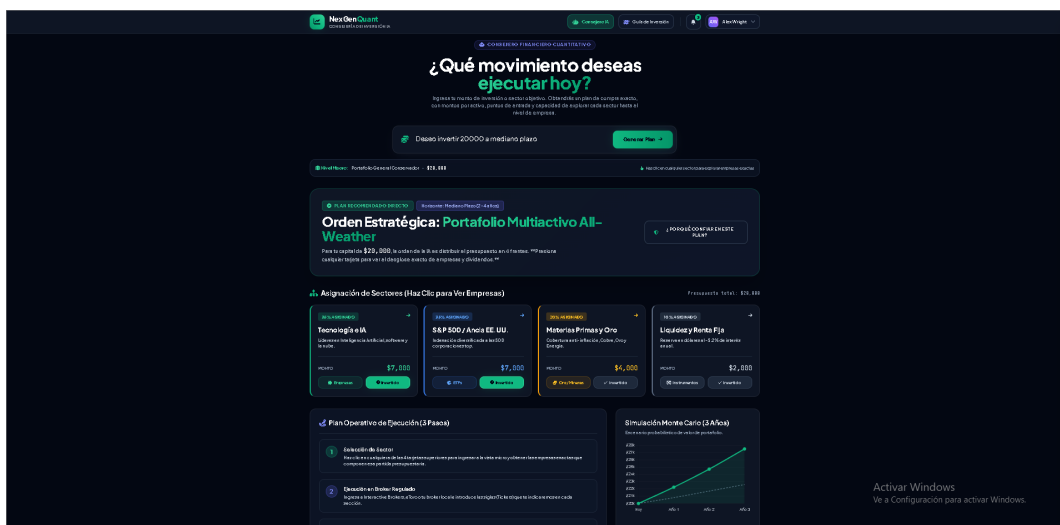


Figura 2: Prototipo 2: Vista de Recomendación de Capital y Asignación Macro/Sectorial

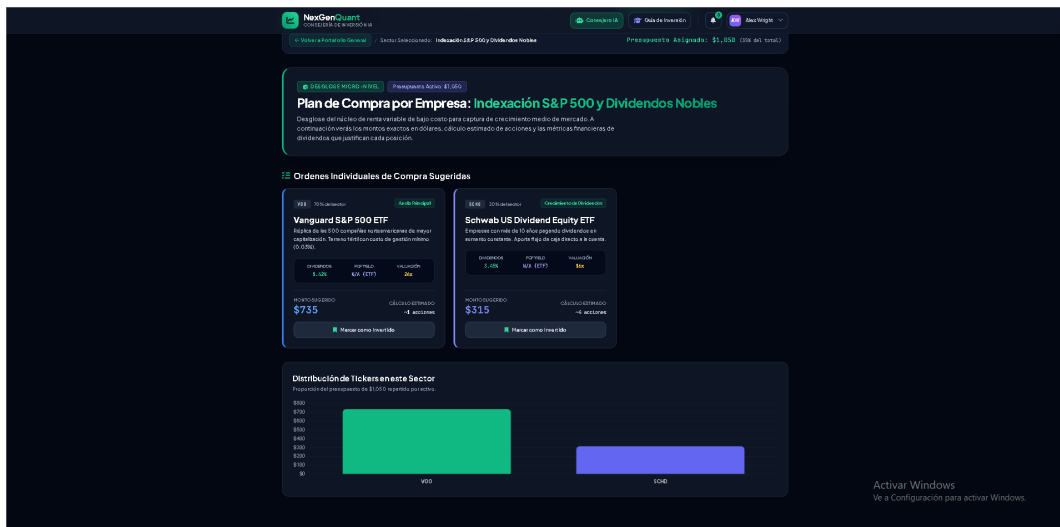


Figura 3: Prototipo 3: Panel Desplegable de Explicabilidad Cualitativa (¿Por qué?)

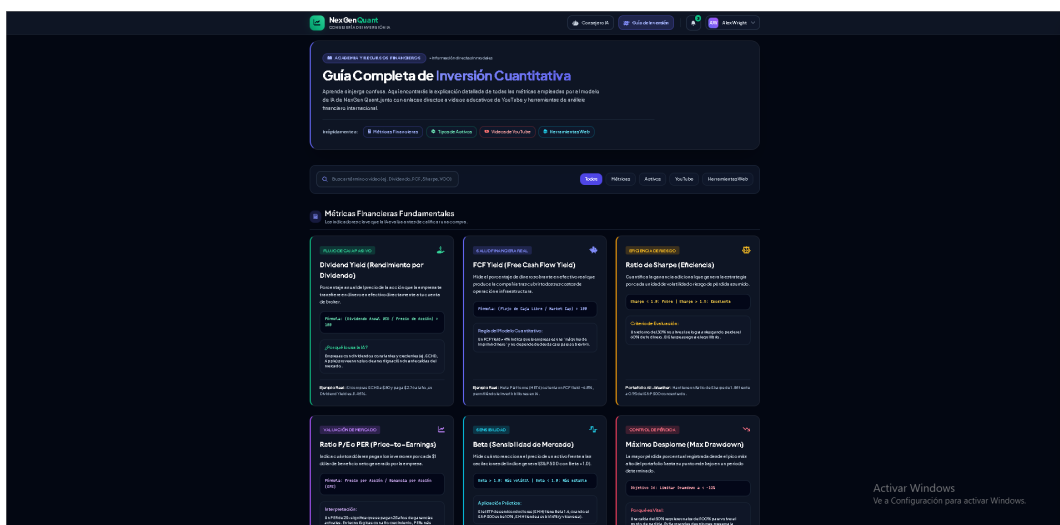


Figura 4: Prototipo 4: Hub Educativo Lateral y Navegación de Glosario

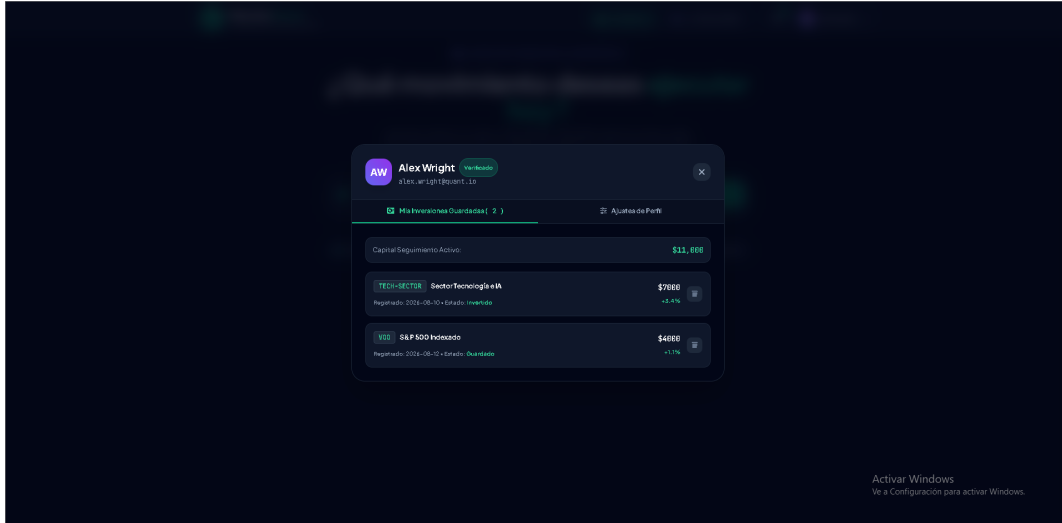


Figura 5: Prototipo 5: Panel de Gestión de Cuenta, Configuración de Tickers e Historial de Alertas

5 Especificación Detallada de Casos de Uso (UML 2.5)

5.1 UC-01 — Ingresar Intención de Inversión

Trazabilidad: HU-3.1 | RF-01, RF-02, RF-03, RF-30, RF-31, RF-32, RF-33 | RNF-04, RNF-16, RNF-19, RNF-24, RNF-33, RNF-36

Actor(es): Usuario Final (Primario).

Disparador (Trigger): El usuario redacta una consulta en lenguaje natural en la barra de búsqueda central y pulsa Enter o clic en buscar.

Precondiciones: Aplicación web cargada en la vista principal (*home*). Acceso público exploratorio.

Flujo Principal:

1. El usuario visualiza la barra de búsqueda conversacional en el centro de la interfaz.
2. El usuario ingresa un prompt en texto libre (ej. *“Invertir 20000”, “Asignar 5000 a tecnología”, “Analizar Apple”*).
3. El usuario envía la consulta.
4. El frontend valida que el texto no esté vacío y despacha la petición al motor clasificador.
5. El sistema clasifica el prompt en una de tres categorías: `INTENT_MACRO`, `INTENT_SECTORIAL` o `INTENT_ACTIVIVO`.
6. El sistema redirige la interfaz a la vista analítica correspondiente.

Flujos Alternos / Excepciones:

- **3a. Prompt no interpretable:** Si la entrada no contiene entidades financieras reconocibles, el sistema despliega una alerta amigable solicitando reformular la consulta sin abandonar la vista.
- **3b. Campo vacío:** La interfaz intercepta el evento en el cliente, bloqueando peticiones HTTP innecesarias.

Postcondiciones: El usuario queda posicionado en la vista analítica adecuada con el contexto procesado.

Reglas de Negocio: La barra conversacional es el canal primario y mandatorio; no existe navegación jerárquica alternativa obligatoria.

Relaciones UML: `«include» UC-02 (Ver Recomendación de Inversión)`.

5.2 UC-02 — Ver Recomendación de Inversión

Trazabilidad: HU-3.9, HU-1.8 | RF-04, RF-17, RF-18, RF-19, RF-20, RF-44, RF-45 | RNF-02, RNF-03, RNF-11, RNF-15, RNF-20, RNF-21, RNF-37

Actor(es): Usuario Final (Primario), API de Predicción / Backend (Secundario / `«system»`).

Disparador (Trigger): Invocación automática tras la clasificación exitosa en UC-01.

Precondiciones: Existencia de un payload de intención clasificado.

Flujo Principal:

1. El frontend envía la solicitud estructurada hacia `POST /api/v1/predict/intent`.
2. El backend procesa la petición contra el pipeline analítico (Qlib $\rightarrow$ MASTER $\rightarrow$ Galformer $\rightarrow$ Z-test con $\alpha = 0,05$).
3. El backend retorna la estructura de recomendación (ponderaciones, activos sugeridos, z -score, p -value y contexto).
4. El frontend renderiza los componentes visuales con los datos reales calculados.

Flujos Alternos / Excepciones:

- **2a. Timeout o caída del backend:** Despliegue de un panel de error legible con botón de reintento, evitando pantallas de carga infinita o errores técnicos crudos.
- **2b. Datos insuficientes para el activo:** Notificación clara cuando el activo posea menos de 30 observaciones históricas o varianza nula.

Postcondiciones: El usuario observa una recomendación fundamentada en datos calculados en vivo.

Reglas de Negocio: Prohibición estricta de datos simulados (*mocks*) en producción.

Puntos de Extensión: *Solicitud de fundamentación analítica* (UC-03), *Recomendación macro o sectorial* (UC-05), *Ajuste manual de asignación* (UC-06), *Inspección de activo individual* (UC-07).

Relaciones UML: Incluido por `«include» UC-01`. `«include» UC-04 (Visualizar Indicador de Confianza)`. Extendido por `«extend» UC-03, UC-05, UC-06, UC-07`.

5.3 UC-03 — Consultar Explicación (“¿Por qué?”)

Trazabilidad: HU-3.2 | RF-06, RF-34, RF-35 | RNF-07, RNF-13, RNF-17, RNF-19, RNF-34

Actor(es): Usuario Final.

Disparador (Trigger): Clic sobre el control interactivo “¿Por qué?” en la tarjeta de recomendación.

Precondiciones: UC-02 ejecutado con éxito; recomendación visible en pantalla.

Flujo Principal:

1. El usuario activa el control “¿Por qué?”.
2. El sistema despliega un panel contextual desplegable (*accordion* o modal ligero).

3. El sistema expone la justificación cualitativa: tendencias intersectoriales y consistencia matemática.
4. El usuario consulta el razonamiento y colapsa el panel.

Postcondiciones: El usuario comprende el sustento del algoritmo sin jerga matemática densa.

Reglas de Negocio: Prohibición absoluta de futuros categóricos (“subirá”, “ganarás”); empleo obligatorio de lenguaje probabilístico y condicional.

Relaciones UML: <<extend>> UC-02.

5.4 UC-04 — Visualizar Indicador de Confianza

Trazabilidad: HU-3.3 | RF-05, RF-09, RF-10, RF-36 | RNF-02, RNF-14, RNF-20, RNF-38

Actor(es): Usuario Final.

Disparador (Trigger): Ejecución automática integrada al renderizado de UC-02.

Precondiciones: Recomendación provista con cálculo de significancia estadística (p -value).

Flujo Principal:

1. El sistema extrae el valor del p -value en el montaje de la vista.
2. El sistema proyecta visualmente el indicador de consistencia porcentual.
3. Se exhibe el texto legal aclaratorio contiguo e inseparable.

Reglas de Negocio: El texto debe aclarar taxativamente que el porcentaje mide consistencia estadística y **no** la probabilidad de ganar dinero.

Relaciones UML: Incluido obligatoriamente por <<include>> UC-02.

5.5 UC-05 — Visualizar Contexto Macroeconómico

Trazabilidad: HU-3.4 | RF-07, RF-37 | RNF-13, RNF-17, RNF-19

Actor(es): Usuario Final.

Disparador (Trigger): Renderizado condicional en pantalla según la intención en UC-02.

Precondiciones: La recomendación generada corresponde a INTENT_MACRO o INTENT_SECTORIAL.

Flujo Principal:

1. El sistema evalúa el tipo de recomendación activa.
2. Se incorpora en pantalla un bloque con tendencias generales, correlaciones intersectoriales de MASTER y factores de riesgo.
3. El usuario asimila el entorno económico que rodea la recomendación.

Relaciones UML: <<extend>> UC-02.

5.6 UC-06 — Ajustar Distribución de Capital

Trazabilidad: HU-3.5 | RF-08, RF-09, RF-38, RF-39 | RNF-09, RNF-12, RNF-16, RNF-22, RNF-38

Actor(es): Usuario Final.

Disparador (Trigger): Manipulación de los controles de asignación porcentual sobre el gráfico Chart.js.

Precondiciones: UC-02 ejecutado; gráfico interactivo visible en pantalla.

Flujo Principal:

1. El usuario visualiza la gráfica de distribución de capital.
2. El usuario altera interactivamente la ponderación de un sector o activo.
3. El sistema valida la restricción de suma cerrada ($\sum \% = 100 \%$).
4. El frontend destruye la instancia gráfica anterior de Chart.js y re-renderiza el nuevo escenario.

Reglas de Negocio: Garantía matemática estricta de suma igual al 100 %. Destrucción de instancias previas para evitar fugas de memoria en el DOM.

Relaciones UML: `<<extend>>` UC-02.

5.7 UC-07 — Visualizar Estrategias de Salida (Stop-loss / Take-profit)

Trazabilidad: HU-3.6 | RF-10, RF-40 | RNF-13, RNF-17, RNF-34, RNF-38

Actor(es): Usuario Final.

Disparador (Trigger): Clic en ver detalle de un activo puntual dentro de la lista de recomendación.

Precondiciones: Recomendación con niveles de mitigación calculados.

Flujo Principal:

1. El usuario expande la ficha técnica del activo.
2. El sistema muestra los niveles sugeridos de Stop-loss y Take-profit.
3. Se incluye una breve explicación accesible del significado de cada umbral.

Relaciones UML: `<<extend>>` UC-02.

5.8 UC-08 — Consultar Hub Educativo

Trazabilidad: HU-3.7 | RF-11, RF-41, RF-42 | RNF-06, RNF-07, RNF-17, RNF-18, RNF-34, RNF-35

Actor(es): Usuario Final.

Disparador (Trigger): Apertura del panel lateral o clic sobre un término técnico interactivo resaltado (ej. *p-value*).

Precondiciones: Ninguna. Componente transversal disponible en toda la sesión.

Flujo Principal:

1. El usuario abre el Hub Educativo.
2. El sistema despliega un panel lateral deslizante (*drawer overlay*) focalizando la definición solicitada.
3. El usuario navega otros conceptos o consulta el glosario alfabético.
4. El usuario cierra el panel lateral.

Reglas de Negocio: La navegación en el glosario no debe recargar la página ni reiniciar el estado o cálculo de la vista analítica.

Relaciones UML: Caso de uso autónomo / independiente.

5.9 UC-09a — Registrarse en la Plataforma

Trazabilidad: HU-2.1 | RF-12, RF-21 | RNF-21, RNF-23, RNF-26, RNF-32

Actor(es): Usuario Final.

Disparador (Trigger): El usuario selecciona “Registrarse” en la barra superior.

Precondiciones: Disponer de un correo electrónico válido no registrado previamente.

Flujo Principal:

1. El usuario ingresa su correo electrónico y contraseña.
2. El sistema valida el formato y verifica unicidad en PostgreSQL.
3. El servicio en Java genera un UUID versión 4 criptográfico.
4. Se persiste el nuevo usuario y se confirma la creación de la cuenta.

Relaciones UML: Caso de uso autónomo del subsistema de gestión de cuentas.

5.10 UC-09b — Configurar Alertas de Ticker

Trazabilidad: HU-2.1, HU-2.2 | RF-13, RF-22, RF-24, RF-25 | RNF-04, RNF-10, RNF-17, RNF-41

Actor(es): Usuario Final.

Disparador (Trigger): El usuario accede a la configuración de alertas en su perfil.

Precondiciones: Usuario autenticado (UC-09a); ticker soportado en el catálogo.

Flujo Principal:

1. El usuario ingresa el `target_ticker` de su interés.
2. El usuario marca la casilla `alerts_enabled = true`.
3. El sistema almacena la preferencia en PostgreSQL y suscribe la regla en la memoria del evaluador Java.

Relaciones UML: Caso de uso autónomo del subsistema de alertas.

5.11 UC-10 — Consultar Historial de Alertas

Trazabilidad: HU-2.3 | RF-14, RF-26, RF-27, RF-48 | RNF-04, RNF-17, RNF-43

Actor(es): Usuario Final.

Disparador (Trigger): El usuario pulsa sobre “Historial de Alertas”.

Precondiciones: Usuario autenticado en el sistema.

Flujo Principal:

1. El frontend invoca `GET /api/v1/users/{id}/alerts`.
2. El backend consulta la tabla `alert_history` en PostgreSQL.
3. El sistema despliega la lista ordenada de forma descendente por fecha (`triggered_at`), mostrando ticker, precio, señal emitida y estampa de tiempo.

Relaciones UML: Caso de uso autónomo del subsistema de alertas.

6 Catálogo de Requerimientos de Software (SRS)

6.1 Requerimientos Funcionales (RF)

Módulo 1: Ingesta y Procesamiento de Datos en Streaming (Python)

- **RF-01:** El sistema deberá recibir ticks de mercado en tiempo real mediante una conexión WebSocket persistente con el proveedor de datos externo. (*Fuente: HU-1.1*)
- **RF-02:** El sistema deberá encolar cada tick recibido en una estructura de datos en memoria (`asyncio.Queue`) en un tiempo menor a 10 ms desde su recepción. (*Fuente: HU-1.1*)
- **RF-03:** El sistema deberá reintentar automáticamente la conexión WebSocket con el proveedor de datos ante una caída, mediante una estrategia de backoff exponencial, sin detener el proceso completo. (*Fuente: HU-1.1*)
- **RF-04:** El sistema deberá descartar y registrar en log cualquier tick recibido con formato inválido, sin interrumpir el pipeline de procesamiento. (*Fuente: HU-1.1*)
- **RF-05:** El sistema deberá transformar los ticks crudos en vectores de Alpha Factors mediante el framework Qlib, persistiendo el resultado en su formato binario nativo. (*Fuente: HU-1.2*)
- **RF-06:** El sistema deberá retornar un estado de “datos insuficientes” cuando existan menos de 30 observaciones históricas disponibles para un ticker, en lugar de ejecutar el cálculo de Alpha Factors. (*Fuente: HU-1.2*)
- **RF-07:** El sistema deberá ejecutar el modelo MASTER sobre los Alpha Factors de múltiples activos, generando una predicción de tendencia por activo junto con su score de correlación inter-sectorial. (*Fuente: HU-1.3*)
- **RF-08:** El sistema deberá ejecutar el modelo Galformer sobre la serie temporal de un activo individual, generando una predicción puntual de tendencia con su horizonte temporal asociado. (*Fuente: HU-1.4*)
- **RF-09:** El sistema deberá aplicar una prueba de hipótesis estadística (Z -test) con nivel de significancia $\alpha = 0,05$ sobre cada predicción generada. (*Fuente: HU-1.5*)
- **RF-10:** El sistema deberá emitir una señal BUY o SELL, junto con su p -value, únicamente cuando el valor absoluto del z -score supere el z crítico de 1.96. (*Fuente: HU-1.5*)
- **RF-11:** El sistema deberá emitir una señal HOLD y omitir la publicación de cualquier evento cuando el z -score se encuentre dentro del rango $[-1,96, 1,96]$. (*Fuente: HU-1.5*)
- **RF-12:** El sistema deberá retornar una señal HOLD sin generar error cuando la desviación estándar del histórico de precios sea igual a cero. (*Fuente: HU-1.5*)
- **RF-13:** El sistema deberá publicar cada señal validada ($p < 0,05$) en el canal Redis `market_signals`, con el payload `{ticker, price, signal, p-value}`, en un tiempo menor a 100 ms desde la recepción del tick original. (*Fuente: HU-1.6*)
- **RF-14:** El sistema deberá registrar en log cualquier fallo de publicación hacia Redis sin bloquear el procesamiento de nuevos ticks entrantes. (*Fuente: HU-1.6*)
- **RF-15:** El sistema deberá insertar en la colección `market_ticks_and_features` de MongoDB un documento por cada tick procesado, incluyendo ticks crudos, Alpha Factors, inferencias de

MASTER y Galformer, p -value y marca temporal — haya o no señal significativa. (*Fuente: HU-1.7*)

- **RF-16:** El sistema deberá garantizar que la colección `market_ticks_and_features` opere bajo un patrón *append-only*, sin permitir operaciones de actualización o eliminación sobre los documentos existentes. (*Fuente: HU-1.7*)
- **RF-17:** El sistema deberá exponer un endpoint POST `/api/v1/predict/intent` que reciba un prompt en texto plano y lo clasifique en una de las categorías `INTENT_MACRO`, `INTENT_SECTORIAL` o `INTENT_ACTIVIVO`. (*Fuente: HU-1.8*)
- **RF-18:** El sistema deberá retornar, para un prompt clasificado como `INTENT_MACRO`, una distribución de capital sugerida a nivel de mercado general. (*Fuente: HU-1.8*)
- **RF-19:** El sistema deberá retornar, para un prompt clasificado como `INTENT_SECTORIAL`, un conjunto de activos pertenecientes al sector identificado en el prompt. (*Fuente: HU-1.8*)
- **RF-20:** El sistema deberá solicitar una aclaración al usuario, en lugar de retornar un error HTTP 500, cuando el prompt recibido no pueda ser clasificado en ninguna de las categorías de intención soportadas. (*Fuente: HU-1.8*)

Módulo 2: Gestión de Usuarios y Alertas (Java)

- **RF-21:** El sistema deberá permitir el registro de un nuevo usuario mediante un correo electrónico válido, generando un identificador único (UUID) para cada registro. (*Fuente: HU-2.1*)
- **RF-22:** El sistema deberá permitir a un usuario registrado configurar un ticker de interés (`target_ticker`) y activar o desactivar la recepción de alertas (`alerts_enabled`), persistiendo dicha configuración. (*Fuente: HU-2.1*)
- **RF-23:** El sistema deberá suscribirse al canal Redis `market_signals` para recibir de forma reactiva las señales publicadas por el contexto analítico. (*Fuente: HU-2.2*)
- **RF-24:** El sistema deberá evaluar cada señal entrante contra las configuraciones activas de los usuarios, notificando únicamente cuando el ticker de la señal coincida con el `target_ticker` configurado y `alerts_enabled` sea verdadero. (*Fuente: HU-2.2*)
- **RF-25:** El sistema deberá omitir la generación de notificación e historial cuando la configuración del usuario tenga `alerts_enabled` en falso, o cuando el ticker de la señal no coincida con su configuración. (*Fuente: HU-2.2*)
- **RF-26:** El sistema deberá insertar un registro en `alert_history` (`ticker`, `price`, `triggered_at`) cada vez que se dispare una alerta válida para un usuario. (*Fuente: HU-2.3*)
- **RF-27:** El sistema deberá permitir a un usuario consultar su historial de alertas, retornando los registros ordenados de forma descendente por fecha de disparo. (*Fuente: HU-2.3*)
- **RF-28:** El sistema deberá garantizar que el paquete `domain` de la aplicación Java no dependa de ninguna librería de infraestructura (Micronaut, JPA, Jackson, Redis). (*Fuente: HU-2.4*)
- **RF-29:** El sistema deberá permitir la ejecución completa de las pruebas unitarias de la lógica de negocio (`SignalNotificationService`) utilizando exclusivamente repositorios *fake* en memoria, sin requerir PostgreSQL ni Redis en ejecución. (*Fuente: HU-2.4*)

Módulo 3: Interfaz Conversacional Basada en Intenciones (Frontend)

- **RF-30:** El sistema deberá proveer una barra de búsqueda principal que reciba una intención de inversión escrita en lenguaje natural. (*Fuente: HU-3.1*)
- **RF-31:** El sistema deberá renderizar la vista de asignación macro/diversificada cuando el prompt del usuario contenga un monto sin especificar sector o activo. (*Fuente: HU-3.1*)
- **RF-32:** El sistema deberá renderizar la vista de activos de un sector cuando el prompt del usuario contenga un monto junto con un sector explícito. (*Fuente: HU-3.1*)
- **RF-33:** El sistema deberá renderizar la vista de análisis individual cuando el prompt del usuario contenga el nombre de un activo específico. (*Fuente: HU-3.1*)
- **RF-34:** El sistema deberá mostrar un elemento interactivo (“¿Por qué?”) que, al ser activado, despliegue una explicación en lenguaje simple sobre la tendencia detectada y la consistencia de la evidencia estadística. (*Fuente: HU-3.2*)
- **RF-35:** El sistema deberá generar todo texto explicativo de recomendaciones utilizando lenguaje condicional/probabilístico, sin incluir en ningún caso verbos en futuro categórico (p. ej. “subirá”, “ganarás”). (*Fuente: HU-3.2*)
- **RF-36:** El sistema deberá mostrar, junto a cada indicador de confianza, un texto aclaratorio explícito indicando que dicho indicador representa la consistencia de la evidencia estadística y no la probabilidad de obtener ganancias. (*Fuente: HU-3.3*)
- **RF-37:** El sistema deberá incluir, en toda vista de recomendación de tipo macro o sectorial, un bloque de contexto macroeconómico con tendencias generales y riesgos asociados. (*Fuente: HU-3.4*)
- **RF-38:** El sistema deberá renderizar un gráfico de distribución porcentual de capital entre activos/sectores para cada recomendación generada a partir de un monto ingresado. (*Fuente: HU-3.5*)
- **RF-39:** El sistema deberá permitir al usuario ajustar manualmente los porcentajes del gráfico de distribución de capital, recalculando y re-renderizando el gráfico con el nuevo escenario. (*Fuente: HU-3.5*)
- **RF-40:** El sistema deberá mostrar, para cada activo recomendado, niveles sugeridos de stop-loss y take-profit acompañados de una breve explicación de su significado. (*Fuente: HU-3.6*)
- **RF-41:** El sistema deberá proveer un panel de Hub Educativo accesible desde cualquier vista de la aplicación, que permita buscar y navegar definiciones de conceptos clave sin interrumpir el flujo principal. (*Fuente: HU-3.7*)
- **RF-42:** El sistema deberá abrir la definición correspondiente en el Hub Educativo cuando el usuario haga clic sobre un término técnico resaltado dentro de una recomendación. (*Fuente: HU-3.7*)
- **RF-43:** El sistema deberá destruir toda instancia previa de gráficos Chart.js antes de renderizar una nueva vista, para prevenir fugas de memoria y congelamiento del navegador. (*Fuente: HU-3.8*)
- **RF-44:** El sistema deberá reemplazar los datos simulados del prototipo por llamadas reales al endpoint `POST /api/v1/predict/intent`, renderizando la respuesta obtenida del backend. (*Fuente: HU-3.9*)

- **RF-45:** El sistema deberá mostrar un estado de error legible en el frontend cuando la llamada al backend falle o exceda el tiempo de espera, en lugar de mantener un estado de carga indefinido. (*Fuente: HU-3.9*)

Módulo 4: Infraestructura y Despliegue (Transversal)

- **RF-46:** El sistema deberá permitir el despliegue simultáneo de FastAPI, Redis, PostgreSQL y MongoDB mediante un único archivo de orquestación (`docker-compose.yml`), quedando todos los servicios disponibles y saludables tras su ejecución. (*Fuente: HU-4.1*)
- **RF-47:** El sistema deberá establecer conexión entre el servicio Java y PostgreSQL de forma automática dentro del entorno orquestado, sin requerir configuración manual adicional. (*Fuente: HU-4.1*)
- **RF-48:** El sistema deberá implementar el adaptador `PostgresSignalRepository` mediante Micronaut Data, persistiendo cada señal procesada y válida en la tabla `alert_history` de PostgreSQL. (*Fuente: HU-4.2*)
- **RF-49:** El sistema deberá garantizar que los tests unitarios de dominio existentes (basados en repositorios fake) continúen pasando sin modificación tras la incorporación del adaptador real de persistencia. (*Fuente: HU-4.2*)

6.2 Requerimientos No Funcionales (RNF)

Rendimiento, Eficiencia y Capacidad

- **RNF-01 (Latencia de Ingesta en Memoria):** El tiempo de procesamiento transcurrido desde la recepción física del tick vía WebSocket hasta su encolado en memoria (`asyncio.Queue`) no debe superar los 10 ms. (*Módulo: Python / HU-1.1*)
- **RNF-02 (Latencia End-to-End del Pipeline):** El tiempo total desde la recepción del tick hasta la emisión de la señal en Redis no debe exceder los 100 ms. (*Módulo: Python / HU-1.6*)
- **RNF-03 (Tiempo de Respuesta Conversacional):** El endpoint `POST /api/v1/predict/intent` debe responder en un percentil 95 (P95) inferior a 1.5 segundos. (*Módulo: Python / HU-1.8, HU-3.9*)
- **RNF-04 (Rendimiento de Evaluación de Alertas):** El evaluador Java debe cotejar una señal contra 1,000 suscripciones en memoria en menos de 50 ms. (*Módulo: Java / HU-2.2*)
- **RNF-05 (Régimen de Escritura Documental):** MongoDB debe soportar al menos 500 documentos por segundo sin degradación de latencia. (*Módulo: Python / HU-1.7*)
- **RNF-06 (Tiempo de Carga Inicial):** Bundle cliente inferior a 350 KB comprimido, logrando $FCP < 1.2s$ y $LCP < 2.5s$. (*Módulo: Frontend / HU-3.1*)

Confiabilidad, Resiliencia y Manejo de Sobrecarga (Backpressure)

- **RNF-07 (Reconexión Resiliente):** Reconexión automática vía backoff exponencial con *jitter* (1s a 60s) ante caídas de WebSocket. (*Módulo: Python / HU-1.1*)
- **RNF-08 (Cola Acotada y Backpressure):** Capacidad máxima de 5,000 eventos en `asyncio.Queue`; política de descarte *drop oldest* con log ante saturación. (*Módulo: Python / HU-1.1, HU-1.2*)

- **RNF-09 (Aislamiento de Fallos en Redis):** Caídas transitorias de Redis no deben bloquear el pipeline de streaming. (*Módulo: Python / HU-1.6*)
- **RNF-10 (Expiración de Mensajes):** TTL máximo de 5 minutos en Redis para prevenir alertas sobre precios desfasados. (*Módulo: Transversal / HU-1.6, HU-2.2*)
- **RNF-11 (Degradación Determinista):** Emisión forzosa de estado HOLD ante muestras inferiores a 30 observaciones o varianza cero. (*Módulo: Python / HU-1.2, HU-1.5*)
- **RNF-12 (Inmutabilidad de Auditoría):** Política estricta *append-only* en MongoDB sin operaciones de modificación ni eliminación. (*Módulo: Python / HU-1.7*)

Arquitectura, Mantenibilidad y Calidad de Código

- **RNF-13 (Aislamiento de Dominio Hexagonal):** Paquete de dominio Java 100 % desacoplado de dependencias de infraestructura. (*Módulo: Java / HU-2.4*)
- **RNF-14 (Testabilidad Pura en Memoria):** Suite de pruebas de dominio ejecutable en menos de 2 segundos sin contenedores activos. (*Módulo: Java / HU-2.4, HU-4.2*)
- **RNF-15 (Estandarización RESTful):** Uso de códigos semánticos HTTP (200, 400, 503) y payloads normalizados en JSON. (*Módulo: Python / HU-1.8, HU-3.9*)
- **RNF-16 (Gestión de Memoria en Frontend):** Destrucción explícita de instancias previas de Chart.js antes de nuevos renderizados. (*Módulo: Frontend / HU-3.8*)
- **RNF-17 (Pool de Conexiones Optimizado):** Conexiones a PostgreSQL gestionadas por HikariCP (máximo 20 conexiones, timeout de 250 ms). (*Módulo: Java / HU-4.2*)
- **RNF-18 (Apagado Seguro - Graceful Shutdown):** Ventana de gracia de hasta 15 segundos para vaciar buffers ante señales SIGTERM/SIGINT. (*Módulo: Transversal / HU-4.1*)

Reglas de Negocio, Ética y Mitigación de Riesgo Legal

- **RNF-19 (Lenguaje Condicional y Probabilístico):** Prohibición absoluta de aseveraciones en futuro categórico (“ganarás”, “subirá”). (*Módulo: Frontend / HU-3.2*)
- **RNF-20 (Transparencia Epistémica del Indicador):** Descargo obligatorio aclarando que la confianza no representa probabilidad de ganancia monetaria. (*Módulo: Frontend / HU-3.3*)
- **RNF-21 (Prohibición de Mocks en Producción):** Todo dato visualizado debe originarse en el cómputo real del pipeline analítico. (*Módulo: Frontend / HU-3.9*)
- **RNF-22 (Integridad en Asignación de Capital):** La interfaz debe garantizar estrictamente que $\sum \% = 100\%$ en la redistribución. (*Módulo: Frontend / HU-3.5*)

Seguridad, Privacidad y Cumplimiento

- **RNF-23 (Identificadores Criptográficos Opacos):** Identificación de usuarios exclusivamente mediante UUID v4. (*Módulo: Java / HU-2.1*)
- **RNF-24 (Sanitización de Entradas):** Validación exhaustiva de prompts en lenguaje natural contra inyecciones y caracteres malformados. (*Módulo: Python / HU-1.8*)
- **RNF-25 (Cifrado de Tráfico en Tránsito):** Cifrado obligatorio TLS 1.3 sobre canales WSS:// y HTTPS://. (*Módulo: Transversal / HU-1.1, HU-3.9*)

- **RNF-26 (Protección de Datos Personales):** Cifrado en reposo para correos y almacenamiento seguro con salting. (*Módulo: Java / HU-2.1*)
- **RNF-27 (Rate Limiting en Inferencia):** Límite máximo de 10 peticiones por minuto por IP/usuario en POST /api/v1/predict/intent. (*Módulo: Python / HU-1.8, HU-3.9*)
- **RNF-28 (Cabeceras Web de Seguridad):** Emisión estricta de políticas CSP, X-Frame-Options (DENY) y X-Content-Type-Options. (*Módulo: Frontend / HU-3.1*)

Gobernanza de Inteligencia Artificial y MLOps

- **RNF-29 (Trazabilidad y Versionado de Modelos):** Almacenamiento del commit hash y versión de los modelos en cada documento de inferencia. (*Módulo: Python / HU-1.3, HU-1.4, HU-1.7*)
- **RNF-30 (Actualización Zero-Downtime):** Recarga de pesos de redes neuronales en caliente sin interrupción de conexiones WebSocket. (*Módulo: Python / HU-1.3, HU-1.4*)
- **RNF-31 (Monitoreo de Data Drift):** Cálculo periódico de divergencias estadísticas entre factores de entrada y sets de entrenamiento. (*Módulo: Python / HU-1.2, HU-1.5*)
- **RNF-32 (Aislamiento de Cómputo Neuronal):** Inferencia ejecutada en workers independientes sin penalizar el event loop asíncrono. (*Módulo: Python / HU-1.3, HU-1.4*)

Usabilidad, Ergonomía y Accesibilidad (UX/UI)

- **RNF-33 (Punto Único Conversacional):** La barra de búsqueda centralizada es la vía primaria de acceso a recomendaciones. (*Módulo: Frontend / HU-3.1*)
- **RNF-34 (Ocultamiento de Complejidad):** Métricas estadísticas crudas confinadas a paneles opcionales secundarios. (*Módulo: Frontend / HU-3.2, HU-3.6*)
- **RNF-35 (Persistencia de Contexto en Glosario):** Apertura del Hub Educativo en overlay sin recargar ni alterar el estado analítico. (*Módulo: Frontend / HU-3.7*)
- **RNF-36 (Validación de Formularios en Cliente):** Interceptación de campos vacíos en el navegador sin invocar la red. (*Módulo: Frontend / HU-3.1*)
- **RNF-37 (Tratamiento Semántico de Errores):** Presentación de mensajes humanizados con opción de reintento guiado. (*Módulo: Frontend / HU-3.9*)
- **RNF-38 (Accesibilidad y Contraste WCAG 2.1 AA):** Relación de contraste mínima de 4.5:1 en señales, badges y gráficas. (*Módulo: Frontend / HU-3.3, HU-3.5, HU-3.6*)
- **RNF-39 (Diseño Adaptable):** Compatibilidad fluida desde pantallas móviles (375px) hasta monitores 4K. (*Módulo: Frontend / HU-3.1, HU-3.5*)

Operabilidad, Telemetría y DevOps

- **RNF-40 (Despliegue Determinista):** Orquestación completa y reproducible de los 5 contenedores mediante docker-compose.yml. (*Módulo: Infraestructura / HU-4.1*)
- **RNF-41 (Verificación de Salud):** Chequeos automatizados (HEALTHCHECK) condicionando el arranque de servicios dependientes. (*Módulo: Infraestructura / HU-4.1*)

- **RNF-42 (Logging Estructurado):** Bitácoras estructuradas en JSON con marca temporal ISO-8601 y nivel de severidad. (*Módulo: Transversal / HU-1.1, HU-1.4, HU-1.6*)
- **RNF-43 (Trazabilidad Distribuida):** Propagación de Correlation-ID a través de Python, Redis, Java y PostgreSQL. (*Módulo: Transversal / HU-1.1, HU-1.6, HU-2.2, HU-4.2*)
- **RNF-44 (Exposición de Métricas Prometheus):** Endpoints `/metrics` publicando lag de bucle, profundidad de colas y latencia P99. (*Módulo: Transversal / HU-4.1*)
- **RNF-45 (Objetivos de Respaldo RPO / RTO):** RPO inferior a 1 hora y RTO inferior a 30 minutos sobre los motores de base de datos. (*Módulo: Infraestructura / HU-4.1, HU-4.2*)

6.3 Matriz de Trazabilidad Requerimientos vs. Historias de Usuario

Tabla 1: Consolidado de Trazabilidad entre Historias de Usuario y Requerimientos

Módulo del Sistema	Historias (HU)	Requisitos RF	Requisitos RNF
Ingesta y Streaming (Python)	HU-1.1 al HU-1.8	RF-01 al RF-20	RNF-01 al RNF-03, RNF-05, RNF-07 al RNF-12, RNF-29 al RNF-32
Gestión y Alertas (Java)	HU-2.1 al HU-2.4	RF-21 al RF-29	RNF-04, RNF-10, RNF-13, RNF-14, RNF-17, RNF-23, RNF-26
Interfaz Usuario (Frontend)	HU-3.1 al HU-3.9	RF-30 al RF-45	RNF-06, RNF-16, RNF-19 al RNF-22, RNF-33 al RNF-39
Infraestructura Transversal	HU-4.1 al HU-4.2	RF-46 al RF-49	RNF-18, RNF-25, RNF-40 al RNF-45

7 Modelado Estructural y del Comportamiento (Diagramas UML / DER)

7.1 Diagrama de Casos de Uso

El diagrama de casos de uso representa las interacciones del Usuario Final y la API de Predicción con los tres subsistemas: el flujo transaccional de recomendaciones, el Hub Educativo transversal y el módulo de gestión de cuenta y alertas.

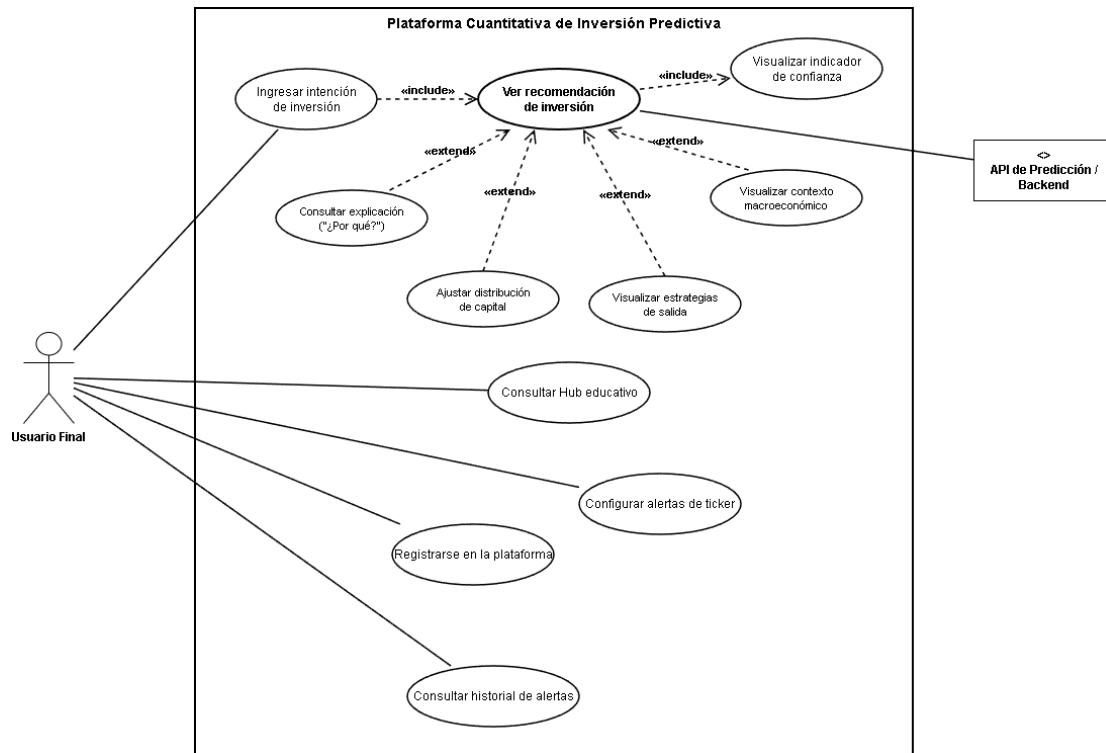


Figura 6: Diagrama de Casos de Uso del Sistema

7.2 Diagrama de Contexto del Sistema

Ilustra los límites operativos del sistema MPS frente a las entidades externas: el proveedor de datos de mercado vía WebSocket, los clientes navegadores y los servicios de autenticación y persistencia.

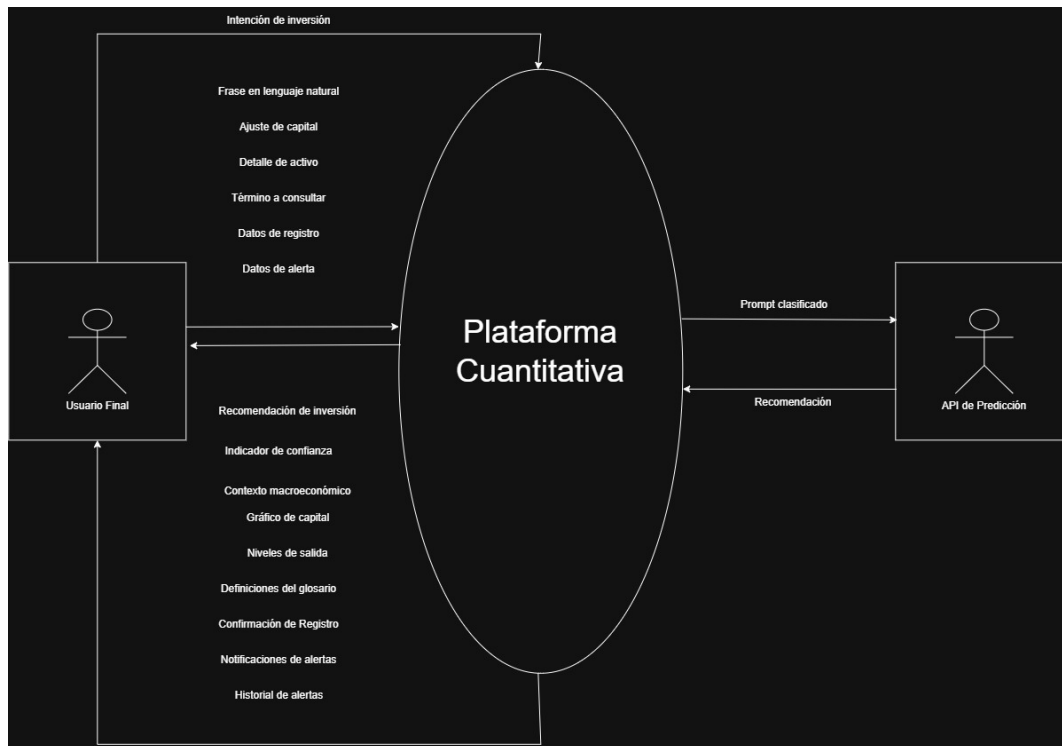


Figura 7: Diagrama de Contexto del Sistema

7.3 Diagramas de Secuencia

Se desglosa la comunicación inter-componentes y el ciclo temporal de invocación a través de las capas de Frontend, Backend Java, FastAPI y Bases de Datos.

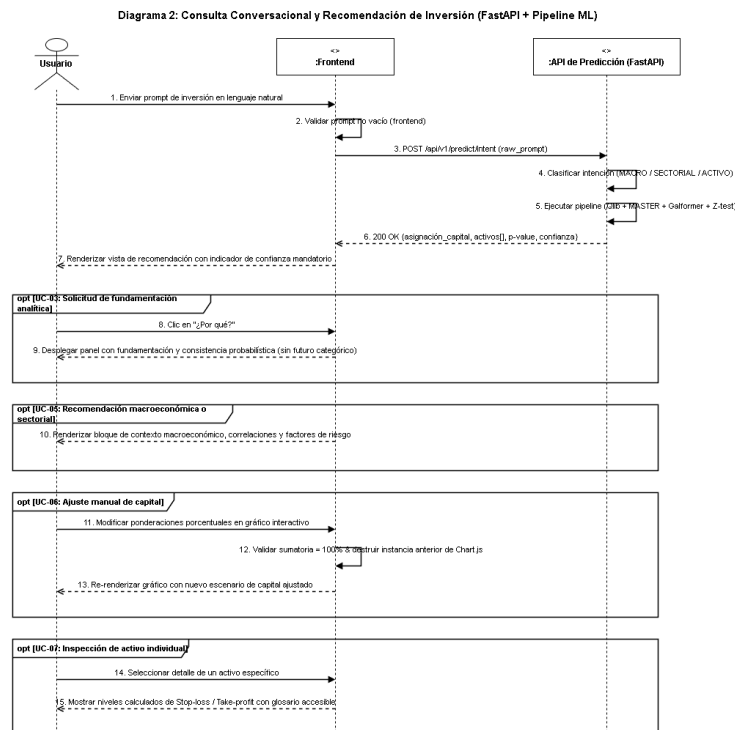
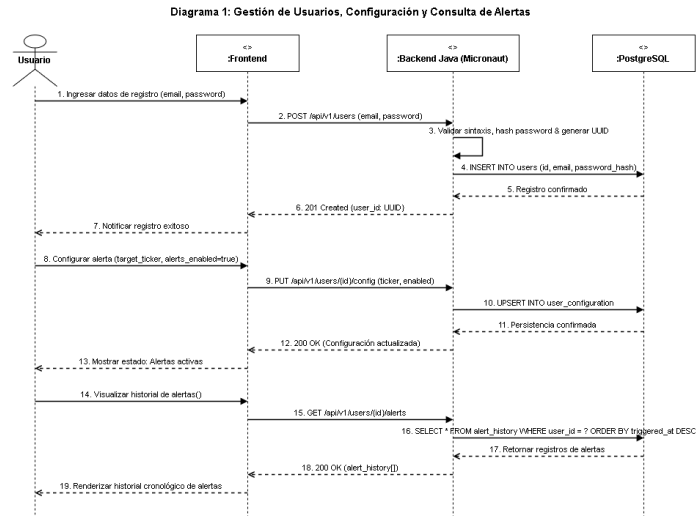


Diagrama 3: Consulta del Hub Educativo (Glosario Transversal sin Recarga de Vista)

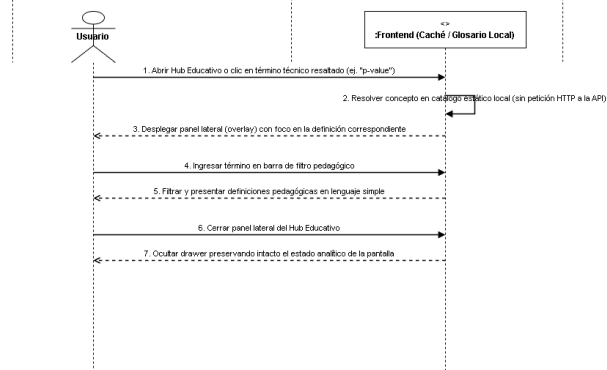


Figura 8: Diagrama de Secuencia de Gestión de Alertas y Recomendación (diagrama_secuencia.png)

7.4 Diagramas de Flujo de Proceso

Modelan el recorrido lógico algorítmico para la clasificación del prompt en lenguaje natural, el pipeline de cálculo cuantitativo y la validación estadística de hipótesis.

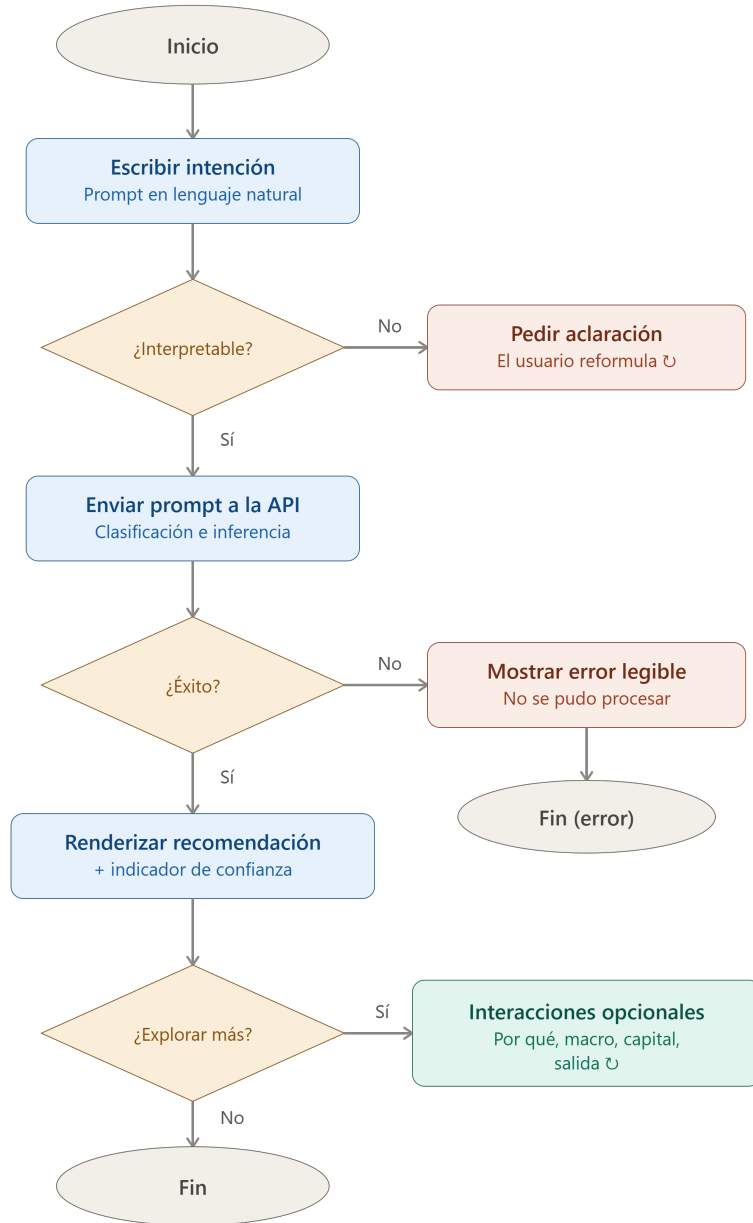


Figura 9: Diagrama de Flujo 1: Clasificación de Intenciones y Enrutamiento

7.5 Modelo Entidad-Relación (MER)

El Modelo Entidad-Relación conceptual clásico (Notación de Chen) delimita las entidades relacionales (usuarios, suscripciones, activos, alertas) y la persistencia documental políglota para el

The diagram illustrates the data model of the Market Data Analysis System. It consists of several entities and their relationships:

- USER** (Entity): Attributes include id, email, password_hash, and created_at. It is connected to **INTENT_PREDICTION_LOG** (1:M), **ASSET** (1:M), and **ALERT_HISTORY** (1:M).
- INTENT_PREDICTION_LOG** (Entity): Attributes include id, timestamp, raw_prompt, parsed_sector, classified_intent, and parsed_amount. It is connected to **USER** (1:M) and **ASSET** (1:M).
- ASSET** (Entity): Attributes include ticker, name, sector, is_active, and evocation_time_mis. It is connected to **USER** (1:M), **INTENT_PREDICTION_LOG** (1:M), and **MARKET_TICK** (1:M).
- MARKET_TICK** (Entity): Attributes include id and timestamp. It is connected to **ASSET** (1:M), **RAW_MARKET_DATA** (1:M), **OLIB_FEATURES** (1:M), and **MASTER_INFERENCE** (1:M).
- ALERT_HISTORY** (Entity): Attributes include id, price, signal, and triggered_at. It is connected to **USER** (1:M), **ASSET** (1:M), and **INTENT_PREDICTION_LOG** (1:M).
- Relationships** (Diamonds):
 - Register** (1:M) connects **USER** and **INTENT_PREDICTION_LOG**.
 - Define** (1:M) connects **USER** and **ASSET**.
 - Monitor** (1:M) connects **ASSET** and **INTENT_PREDICTION_LOG**.
 - Dispara** (1:M) connects **ASSET** and **ALERT_HISTORY**.
 - Contains** (1:M) connects **MARKET_TICK** and **RAW_MARKET_DATA**.
 - Extract** (1:M) connects **MARKET_TICK** and **OLIB_FEATURES**.
 - Infer** (1:M) connects **MARKET_TICK** and **MASTER_INFERENCE**.
 - Project** (1:M) connects **RAW_MARKET_DATA** and **GALFORMER_INFERENCE**.
 - Valid** (1:M) connects **OLIB_FEATURES** and **STATISTICAL_VALIDATION**.

7.6 Diagrama de Clases (UML)

The diagram illustrates the data architecture for the Market Analysis Pipeline, divided into two main sections: RDBMS and Document Store.

RDBMS — PostgreSQL (Gestión de Usuarios, Preferencias y Alertas)

- USER**
 - [PK] id : UUID
 - * email : VARCHAR(255)
 - * password_hash : VARCHAR(255)
 - * created_at : TIMESTAMP
- ASSET**
 - [PK] ticker : VARCHAR(20)
 - * name : VARCHAR(200)
 - * sector : VARCHAR(50)
 - * is_active : BOOLEAN
- USER_CONFIGURATION**
 - [PK] id : UUID
 - [FK] user_id : UUID
 - [FK] target_ticker : VARCHAR(20)
 - * alerts_enabled : BOOLEAN
 - * created_at : TIMESTAMP
- ALERT_HISTORY**
 - [PK] id : UUID
 - [FK] user_id : UUID
 - [FK] ticker : VARCHAR(20)
 - * price : DECIMAL(10,4)
 - * signal : VARCHAR(50)
 - * triggered_at : TIMESTAMP

DOCUMENT STORE — MongoDB (Audit Logs & Pipeline Analytical Streaming)

- INTENT_PREDICTION_LOG**
 - [PK] id : OBJECT_ID
 - [REF] user_id : UUID (nullable)
 - * timestamp : TIMESTAMP
 - * raw_prompt : TEXT
 - * classified_intent : VARCHAR(50)
 - * parsed_amount : DOUBLE
 - * parsed_sector : VARCHAR(50)
 - * parsed_ticker : VARCHAR(20)
 - * execution_time_ms : INTEGER
- MARKET_TICK (market_ticks_and_features)**
 - [PK] id : OBJECT_ID
 - [FK] ticker : VARCHAR(20)
 - * timestamp : TIMESTAMP
 - .. Indices: (ticker_id, timestamp-1) ..
- RAW_MARKET_DATA** (Embedded in MARKET_TICK)
 - * price : DOUBLE
 - * volume : DOUBLE
- QLB_FEATURES** (Embedded in MARKET_TICK)
 - * feature_vector : BINARY (alpha)
- MASTER_INFERENCE** (Embedded in MARKET_TICK)
 - * raw_tensors : BINARY
 - * trend : VARCHAR(20)
- GALFORMER_INFERENCE** (Embedded in MARKET_TICK)
 - * raw_tensors : BINARY
 - * projected_horizon : DOUBLE
- STATISTICAL_VALIDATION** (Embedded in MARKET_TICK)
 - * z_score : DOUBLE
 - * p_value : DOUBLE
 - * is_significant : BOOLEAN
 - * final_signal : VARCHAR(50)

Data Flow:

- write_user_prefs (FN)**: RDBMS USER to DOCUMENT STORE MARKET_TICK
- read_user_prefs (FN)**: DOCUMENT STORE MARKET_TICK to RDBMS USER
- define (FN)**: RDBMS USER_CONFIGURATION to RDBMS USER
- monitoring (FN)**: RDBMS ASSET to RDBMS USER_CONFIGURATION
- results (FN)**: RDBMS ALERT_HISTORY to RDBMS USER
- origin (FN)**: RDBMS ASSET to RDBMS ALERT_HISTORY

22